How you should conduct the review

1. Start with the problem — 30 seconds

Say:

“RelayNet is an adaptive emergency mesh-routing prototype designed for disaster environments where conventional communication infrastructure may be unavailable. We model communication between a source and destination using three UAV relay nodes. Instead of selecting a relay using only shortest path or signal strength, RelayNet considers network conditions such as RSSI, battery, queue length, mobility, link stability and hop count.”

Then:

“Our system combines dynamic MANET routing, a temporal Knowledge Graph for contextual reasoning, and independent multi-agent Q-learning for adaptive relay selection.”

That gives them the **problem + contribution** immediately.

2. Show your architecture

Use this mentally as your main explanation:

```
Network State
     ↓
Dynamic MANET
     ↓
Observations
     ↓
Temporal Knowledge Graph
     ↓
Contextual Reasoning
     ↓
Multi-Agent Q-Learning
```

```
     ↓
Relay Selection
     ↓
Packet Forwarding
     ↓
Performance Evaluation
```

Say:

“The MANET provides the dynamic network state. The Knowledge Graph converts observations into contextual information. The Q-learning agents learn from previous forwarding outcomes. Finally, the KG score and learned Q-score are combined to select the relay.”

Then mention:

```
FinalScore = 0.65 KGScore + 0.35 QScore
```

This is one of the most important equations to remember.

3. Immediately demonstrate the dashboard

Don't start by opening source code.

Open:

```
python -m streamlit run dashboard/app.py
```

Then demonstrate in this order:

A. Overview

Show:

- PDR
- packet loss

- delay
- throughput
- energy
- lifetime

Explain:

“These KPIs change according to the selected disaster scenario and routing method.”

B. MANET

Show the topology.

Say:

“This is the dynamic MANET layer. Neighbours are discovered based on wireless range, and multi-hop routes are calculated dynamically.”

C. Knowledge Graph

Show the KG visualization.

Say:

“The KG stores temporal observations as subject-predicate-object triples. It also derives contextual facts such as stable link, low queue and sufficient battery.”

D. Q-Learning

Show:

- parameters
- training graph
- Q-table

Say:

“Each relay maintains an independent Q-table. The agents learn which relay-selection actions produce better forwarding outcomes.”

E. Performance

Show the multi-seed results.

Say:

“We evaluated five scenarios, four methods and ten seeds, giving 200 Python runs.”

F. NS-3 Validation

This is your **new strong feature**.

Say:

“We additionally performed independent NS-3.47 validation using AODV, OLSR and DSR. This is not an implementation of AEMRP inside NS-3; it is an independent MANET baseline.”

That distinction is extremely important.

4. If they ask "What is your main contribution?"

Answer:

“The main contribution is the integration of dynamic network-state observation, temporal contextual reasoning and independent multi-agent learning for adaptive UAV relay selection in a simulated disaster MANET.”

Then add:

“We also evaluated the system statistically across multiple scenarios and seeds and independently validated MANET behavior using NS-3.”

5. If they ask "Why Knowledge Graph?"

Answer:

“The Knowledge Graph provides contextual and explainable reasoning. Instead of only producing a numerical score, it can represent relationships such as a relay having sufficient battery, low queue, stable link and strong RSSI. This makes the relay recommendation more interpretable.”

If they ask:

“Why not just use ML?”

Say:

“Pure learning can make the decision difficult to interpret. The KG provides explicit contextual information, while Q-learning provides adaptation from experience. Combining them gives us both contextual reasoning and learning.”

6. If they ask "Why multi-agent Q-learning?"

Answer:

“Because each UAV relay has its own local state and forwarding decisions. We therefore model each relay as an independent learning agent with its own Q-table.”

Do **not** say that you implemented sophisticated deep MARL.

Your implementation is **independent Q-learning**, not MADDPG.

7. If they ask "Why 0.65 and 0.35?"

This is a likely question.

Answer honestly:

“The current implementation gives 65% weight to contextual KG knowledge and 35% to learned Q-values. This was chosen to keep contextual network knowledge dominant while allowing learned experience to influence the decision. It is a prototype parameter rather than a mathematically optimal value.”

That's a much stronger answer than pretending 0.65/0.35 is theoretically optimal.

8. If they ask about your results

This is where you must be careful.

They may ask:

“Does KG+MARL always perform best?”

Absolutely do not say yes.

Say:

“No. KG+MARL is competitive but does not win in every scenario. Its strongest result is under high mobility, where it achieves the best PDR and throughput among the evaluated methods. Under low-battery conditions, the current reward function needs improvement.”

This demonstrates scientific honesty.

9. If they ask about NS-3

Likely question:

“Is your AEMRP implemented in NS-3?”

Answer:

“No. The Python simulator contains the AEMRP-style KG and multi-agent learning implementation. NS-3 is an independent validation layer where we evaluate AODV, OLSR and DSR under the same broad disaster scenarios.”

Then:

“We deliberately don't claim that KG or Q-learning is running inside NS-3.”

This distinction can actually **increase your credibility**.

10. If they ask why NS-3 results are much lower than Python

Answer:

“The two simulators use different topology, radio, mobility, traffic and packet models, so their absolute PDR values are not directly comparable. NS-3 is being used as independent MANET baseline validation rather than as a numerical continuation of the Python experiment.”

Excellent answer.

11. If they ask "What happens when a relay fails?"

Answer:

“Relay availability is part of the network state. The MANET layer updates available links and routes, and the relay-selection layer avoids unavailable relay candidates. We also have a dedicated relay-failure scenario for evaluation.”

12. If they ask "What happens when battery becomes low?"

Answer:

“Battery contributes to relay selection and energy consumption is tracked. We also have a low-battery scenario. One limitation we identified is that the current KG+MARL reward shaping does not handle low-battery conditions as effectively as the simpler methods, so battery-aware reward optimization is an important future improvement.”

Again: honest and technically defensible.

13. If they ask "Why only three UAVs?"

Say:

“This is a controlled semester-level prototype. Three relay nodes allow us to demonstrate dynamic relay selection, failures and multi-hop behavior without making the simulation unnecessarily complex. The architecture can be extended to larger relay populations.”

14. If they ask "Is this real-world deployable?"

Say:

“Not currently. This is a simulation and research prototype. Physical UAV deployment, real mobile-device communication, Wi-Fi Direct or Bluetooth Mesh and production emergency infrastructure are future work.”

Do **not** oversell it.

15. Your 2-minute emergency demo script

If the faculty says:

“Show us your project.”

Do this:

0:00–0:20

“RelayNet is an adaptive emergency MANET routing prototype using UAV relays.”

Show Overview.

0:20–0:45

Show MANET.

“The topology changes dynamically and routes are discovered based on available wireless links.”

0:45–1:05

Show KG.

“The temporal KG represents relay conditions and derives contextual recommendations.”

1:05–1:25

Show Q-learning.

“Each relay independently learns from forwarding outcomes, and the final score combines KG context with learned Q-values.”

1:25–1:45

Show Performance.

“We evaluated five scenarios, four methods and ten seeds—200 Python runs.”

1:45–2:00

Show NS-3 Validation.

“Finally, we independently validated the MANET behavior with 150 NS-3 runs across AODV, OLSR and DSR.”

Then stop.

Don't start explaining every implementation detail unless they ask.

Most important questions to memorize

Before your review, make sure you can answer these without looking at notes:

Question	Core answer
What problem?	Infrastructure-less disaster communication
What is RelayNet?	Adaptive UAV-relay MANET prototype
Why KG?	Contextual + explainable reasoning
Why Q-learning?	Learn relay decisions from experience
Why multi-agent?	Independent relay agents
Final score?	`0.65 KG + 0.35 Q`
How many Python runs?	200
How many NS-3 runs?	150
Python methods?	Static, Baseline, Contextual KG, KG+MARL
NS-3 protocols?	AODV, OLSR, DSR
Best KG+MARL condition?	High mobility
Does KG+MARL always win?	**No**
Main limitation?	Low-battery reward shaping
Is KG in NS-3?	**No**
Is AEMRP natively in NS-3?	**No**

Neo4j implemented?	**No**
Physical UAVs?	**No**
Main future work?	Direct NS-3 integration, better reward shaping, real deployment

One final piece of advice

Do not try to impress them by claiming more than you built.

Your strongest review position is:

“We built a complete simulation prototype, evaluated it statistically, identified where it works well, identified where it doesn't, and independently validated the MANET baseline in NS-3.”

That's a much more defensible final-year/semester project story than claiming a production-ready AI emergency network.